
KD-SLAM

*One KD-Tree to deskew them all
One KD-Tree to match them
One KD-Tree to close the loops and in the map bind them*

✉ **Giorgio Grisetti** *

Department of Computer, Control and Management Engineering
Sapienza University of Rome
Via Ariosto 25
Rome, I-00185
grisetti@diag.uniroma1.it

July 1, 2026

ABSTRACT

PCA KD-trees are a remarkably versatile structure: equivariant under rigid transformations, they encode geometry at every level from root to leaf. Can a single such structure carry the full LiDAR SLAM pipeline – deskewing, odometry, place recognition, and multi-view registration – without switching representation? This paper is the answer: a composable 2D/3D, CPU/CUDA, rigid and continuous-time mapping suite that runs in real time on a 10 W embedded processor. Evaluated on VBR and KITTI. The code is available at https://github.com/rvp-group/kd_slam2. A video is here <https://youtu.be/c-sCCt9hMmI>.

Keywords SLAM · KD-Trees · Lidar

1 Introduction

This work is a study on the properties of a search structure: the PCA KD-tree, in the context of LiDAR SLAM and 2D/3D reconstruction. To test whether these properties were useful, we assembled a composable 2D/3D, CPU/CUDA, rigid and continuous-time mapping pipeline that addresses online SLAM and offline map refinement.

Leveraging a single simple structure, we address many aspects of the pipeline in a coherent manner: deskewing, odometry, place recognition, and multiview registration all operate on the same representation without switching at any point. The aim is not to chase the last digit of the benchmarks, but to provide a solid proof of concept and a usable system ready to be extended. On a 10 W embedded processor it runs in real time; the bag reader is the bottleneck, not the pipeline. The results sit without shame on the VBR leaderboard; test sequences have been submitted to the KITTI evaluation server. We just did not design for it.

The architecture stems from an effort to unify paradigms and compress knowledge into representation.

The specific contributions are:

1. **Tree morphing for deskewing.** Velocity is applied to search nodes, not points; the search structure is preserved and the approximation is first-order exact.
2. **PCA KD-tree descriptors.** Derived directly from tree geometry, no training, no preprocessing; the match provides the relative transform as a byproduct.
3. **ICP unified with search.** Correspondence finding, Jacobian computation, and normal estimation happen in the same tree walk; symmetric point-to-plane ICP comes for free from the PCA splitting geometry.

4. **Multi-view registration.** Joint optimization over the keyframe co-visibility graph via multi-view ICP factors. In the rigid mode, poses only; in the CT mode, poses and velocities jointly, with deskewing applied per iteration.
5. **Loop consensus.** Spatial / descriptor / ICP verification funnel; loops confirmed only when the spatial χ^2 gate agrees.
6. **One codebase, all paths.** The same C++ template machinery covers 2D/3D, CPU/CUDA, rigid/CT without code duplication.

We provide:

- This paper.
- A C++/CUDA implementation that we believe is rather decent, available at https://github.com/rvp-group/kd_slam2.
- Experiments on VBR [3] and KITTI [15] to ground the claims.

Section 2 surveys related work. Sections 3–6 develop the tree structure and its key properties: construction, normal estimation, deskewing, registration, and place recognition. Section 7 assembles these into a full SLAM pipeline with loop closure. Section 8 adds offline multiview registration over the keyframe graph. Section 9 covers the implementation. Section 10 reports the experiments. Enjoy.

2 Related Work

Smith and Cheeseman [31] formalized the problem of simultaneous localization and mapping in 1987; four decades of progress later, the literature is vast [4]. What follows is the context that came to mind while writing; a comprehensive survey would be a book.

Early solutions relied on 2D laser scanners and graph-based formulations that decoupled the data association problem from the optimization. Foundational work in this era includes the globally consistent scan alignment of Lu and Milios [24] and the incremental mapper of Gutmann and Konolige [21]. With the availability of 3D LiDAR sensors, Nüchter *et al.* [25] were among the first to demonstrate 6-DOF LiDAR SLAM in outdoor environments, establishing the scan-matching-plus-pose-graph paradigm that most subsequent work follows to this day.

Point cloud registration. Geometric registration of point clouds traces back to the Iterative Closest Point algorithm of Besl and McKay [1]. Despite its age, ICP remains a competitive baseline. Its variants – point-to-plane, plane-to-plane, and their probabilistic generalizations such as GICP [28] (Segal *et al.*) – improved convergence and robustness by modelling each point as a Gaussian whose covariance is estimated from its local neighbourhood. Pomerleau *et al.* [27] provide a systematic comparison of ICP variants on real-world data, establishing a useful baseline for understanding where each variant succeeds and fails.

Biber and Straßer [2] introduced the Normal Distributions Transform, compressing point clouds into voxel-level Gaussian distributions, trading point-level detail for a smoother objective landscape; Stoyanov *et al.* [32] refined this idea into a fast and accurate scan registration framework that demonstrated NDT could be both principled and practical. Deschaud [11] pushed scan-to-model matching with an implicit surface formulation (IMLS-SLAM), achieving high accuracy at higher computational cost.

Search structures. Axis-aligned KD-trees offer logarithmic nearest-neighbor search but assign normals by brute force over leaf points, divorced from the tree geometry. PCA-based splitting – orienting the partition plane along the axis of maximum spread – was known early but underexplored. In 3D, PCA reduces to a 3×3 eigendecomposition and costs nothing; the splitting geometry then carries enough information to support normal estimation, deskewing, and descriptor extraction as direct byproducts of tree construction. This observation is the starting point of the present work.

LiDAR odometry and SLAM. Zhang and Singh [35] (LOAM) extracted edge and planar features from LiDAR scans and matched them across frames, achieving state-of-the-art accuracy on KITTI at the cost of a feature extraction stage that commits to a geometric abstraction and discards raw point data. Shan and Englot [30] (LeGO-LOAM) added ground segmentation, limiting applicability to outdoor ground vehicles.

The return to simple, direct ICP without feature preprocessing was advocated convincingly by Vizzo *et al.* [34] (KISS-ICP), which demonstrated that point-to-point ICP with adaptive thresholding and voxelization is competitive with much more complex pipelines. Guadagnino *et al.* [20] (KISS-SLAM) extended this philosophy to a complete SLAM pipeline:

KISS-ICP odometry, local map construction, loop closure via ORB descriptors projected onto a 2D density image, and pose graph optimization. KISS-SLAM demonstrates that a clean pipeline of well-chosen components achieves state-of-the-art performance with minimal tuning.

Pan *et al.* [26] (PIN-SLAM) integrate LiDAR odometry with a probability-based implicit neural map, achieving accurate reconstruction and loop closure without pose graph optimization; the computational load is high and real-time operation is not yet demonstrated.

Continuous-time methods. Droschel and Behnke [13] proposed continuous-time SLAM modeling the sensor trajectory as a smooth curve to compensate for motion during acquisition. Dellenbach *et al.* [10] (CT-ICP) applied this idea specifically to LiDAR odometry, recovering per-point timestamps and deskewing point clouds at registration time. The improvement is real on high-speed or rotating platforms. Cao *et al.* [5] (RESPLE) take an alternative formulation: the trajectory is a B-spline and the control points are the state, estimated recursively via an iterated extended Kalman filter. The same motion-distortion idea extends naturally to radar: DRO [23] corrects Doppler distortion in spinning FMCW radar scans using a gyroscope, achieving accurate odometry without feature extraction.

Place recognition. Kim and Kim [22] (Scan Context) encode the spatial distribution of points in a rotation-invariant polar representation, enabling efficient retrieval without learning. Uy and Lee [33] (PointNetVLAD) and Dubé *et al.* [14] (SegMatch) achieve higher discriminability through learned representations at the cost of requiring task-specific training data.

Own prior work. Normal-based ICP [29] proposed dense registration using per-point covariances derived from local PCA, anticipating the tree-based covariance structure used here. Building the covariance structure from scratch at every frame was expensive; integrating it into a persistent search structure was the missing piece.

Multi-cue photometric registration (MPR) [7] extended NICEP with intensity and photometric channels alongside geometry. The key idea is that data association is implicit: the objective is a photometric consistency term over all cues jointly, with no explicit correspondence step. Di Giammarino *et al.* [18] exploited LiDAR intensity for place recognition, demonstrating that the information already present in the point cloud is sufficient for loop detection without separate cameras or learned descriptors. MD-SLAM [12] built on both [7, 18] to assemble a full direct SLAM pipeline working with RGB-D and LiDAR sensors via the same photometric objective, with no sensor-specific adaptation.

Photometric approaches can be extremely accurate but suffer from a narrow convergence basin, only partially addressed by multi-resolution schemes – hence the transition to point clouds. MAD-ICP [16] addressed robustness by improving the matching strategy and adding a velocity model, but remained open-loop. MAD-BA [8] closed that gap with a full bundle adjustment that jointly optimizes sensor poses and map structure as surfels, incorporating a generalized LiDAR uncertainty model.

Splat-LOAM [17] uses Gaussian primitives as the sole scene representation, driven exclusively from LiDAR measurements via spherical projection; odometry and mapping share the same representation, with SOTA mapping quality and minimal GPU requirements. De Rebotti *et al.* [9] take posed clouds as input and build a variance-adaptive voxel grid stored in a flat spatial hash table – constant-time GPU access, no octree overhead – that dynamically adjusts resolution based on local SDF variance and supports Gaussian splatting rendering as a byproduct.

Ceriola *et al.* [6] (G-solver) combine Gaussian Belief Propagation with Gaussian Process motion priors for continuous-time trajectory estimation, handling heterogeneous asynchronous sensors – rolling shutter, LiDAR, radar, event cameras – in a distributed setting without specialized synchronization.

KD-SLAM is a study: one PCA KD-tree structure, no representation switch, from raw scan to bundle-adjusted map, on any hardware – a foundation to build from, not a claim on the last digit.

3 KD-Tree Construction

A KD-Tree is a search structure obtained by recursively partitioning reference points with a hyperplane. The hyperplane is chosen to split the data roughly in two halves. Common approaches choose a hyperplane orthogonal to one of the coordinate axes. In a KD-Tree, the search for a query point $\mathbf{q}$ proceeds from the root to the leaves. When a non-leaf node N with splitting hyperplane π is traversed, the search continues along the child of N that lies on the same side of π as $\mathbf{q}$.

This greedy search is neither correct nor complete, and it can return a point that is not the closest one in the set, but behaves reasonably well in practice. Choosing a “wrong” branch may result in a suboptimal answer. A necessary

condition for incompleteness to arise is that the search ball around the query point intersects a traversed hyperplane. Both completeness and correctness can be recovered by spawning a search on both branches when this situation occurs.

Intuitively, in the construction of the KD-Tree we want to select the plane that stays as far as possible from each point while passing through the mean of the point set, keeping the tree balanced.

If we allow hyperplanes to be oriented freely we gain an additional degree of freedom that can both capture the variable-resolution statistics of our point distribution and yield a plane more likely to stay away from the points. To this end, a straightforward option is to compute the eigenvalue decomposition of the point distribution and choose as splitting plane the one that passes through the mean and whose normal is parallel to the largest eigenvector.

The procedure terminates when some criterion is met, and each leaf can hold a small set of points on which brute-force search is convenient. The eigenvalue decomposition is repeated at every node and can be used to characterize the surface normal when computed on the points stored in the leaves: the normal is the smallest eigenvector of the point covariance.

Using a non-trivial splitting policy—such as a termination criterion based on point density—we can achieve a regularization that resembles voxelization.

PCA-based KD-Trees were introduced in the 1980s, but the prohibitive cost of computing PCA in high dimensions drove the community towards axis-aligned methods. For geometric data such as 2D/3D points, however, PCA can be computed efficiently and closed-form solutions exist. Being completely data-driven, PCA allows us to transfer operators directly from the points to the KD-Tree.

In Alg. 1 we sketch a recursive version of KD-Tree construction, while in Alg. 2 and Alg. 3 we show the fast (incomplete) leaf search and the recursive complete point search.

Algorithm 1 Recursive PCA KD-Tree Construction

Require: point set $P \subset \mathbb{R}^N$
Ensure: tree node $\langle \mathbf{m}, \mathbf{d}, T_l, T_r \rangle$

```

1: function BUILDTREE( $P$ )
2:    $\mathbf{m} \leftarrow \frac{1}{|P|} \sum_{\mathbf{p} \in P} \mathbf{p}$ 
3:    $\Sigma \leftarrow \frac{1}{|P|} \sum_{\mathbf{p} \in P} \mathbf{p}\mathbf{p}^\top - \mathbf{m}\mathbf{m}^\top$ 
4:    $\Lambda, \mathbf{U} \leftarrow \text{eig}(\Sigma)$  ▷ columns of  $\mathbf{U}$  sorted by eigenvalue,  $\mathbf{u}_{\min}$  smallest,  $\mathbf{u}_{\max}$  largest
5:   if ISLEAF( $P, \mathbf{m}, \Sigma$ ) then
6:     return  $\langle \mathbf{m}, \mathbf{u}_{\min}, \emptyset, \emptyset \rangle$  ▷ leaf:  $\mathbf{d}$  is surface normal
7:   end if
8:    $P_l \leftarrow \{\mathbf{p} \in P \mid (\mathbf{p} - \mathbf{m}) \cdot \mathbf{u}_{\max} < 0\}$ 
9:    $P_r \leftarrow \{\mathbf{p} \in P \mid (\mathbf{p} - \mathbf{m}) \cdot \mathbf{u}_{\max} \geq 0\}$ 
10:  return  $\langle \mathbf{m}, \mathbf{u}_{\max}, \text{BUILDTREE}(P_l), \text{BUILDTREE}(P_r) \rangle$ 
11: end function

```

Algorithm 2 KD-Tree Leaf Search (fast/complete)

Require: tree $\mathcal{T}$, query point $\mathbf{q}$, ball radius R , flag *complete*
Ensure: set of candidate leaves $\mathcal{L}$

```

1: function SEARCHLEAVES( $\mathcal{T}, \mathbf{q}, R, \text{complete}$ )
2:   if ISLEAF( $\mathcal{T}$ ) then
3:     return  $\{\mathcal{T}\}$ 
4:   end if
5:    $r \leftarrow (\mathbf{q} - \mathcal{T}.\mathbf{m}) \cdot \mathcal{T}.\mathbf{d}$ 
6:   if complete and  $|r| < R$  then ▷ query ball straddles the splitting plane
7:     return SEARCHLEAVES( $\mathcal{T}.T_l, \mathbf{q}, R, \text{complete}$ )  $\cup$  SEARCHLEAVES( $\mathcal{T}.T_r, \mathbf{q}, R, \text{complete}$ )
8:   else if  $r \geq 0$  then
9:     return SEARCHLEAVES( $\mathcal{T}.T_r, \mathbf{q}, R, \text{complete}$ )
10:  else
11:    return SEARCHLEAVES( $\mathcal{T}.T_l, \mathbf{q}, R, \text{complete}$ )
12:  end if
13: end function

```

Algorithm 3 KD-Tree Point Search**Require:** tree $\mathcal{T}$, query point $\mathbf{q}$, ball radius R , flag *complete***Ensure:** set of points P_{out} within distance R of $\mathbf{q}$

```

1: function SEARCHPOINTS( $\mathcal{T}$ ,  $\mathbf{q}$ ,  $R$ , complete)
2:    $\mathcal{L} \leftarrow \text{SEARCHLEAVES}(\mathcal{T}, \mathbf{q}, R, \text{complete})$ 
3:    $P_{\text{out}} \leftarrow \emptyset$ 
4:   for all  $\ell \in \mathcal{L}$  do
5:      $P_{\text{out}} \leftarrow P_{\text{out}} \cup \text{BRUTEFORCE}(\ell, \mathbf{q}, R)$ 
6:   end for
7:   return  $P_{\text{out}}$ 
8: end function

```

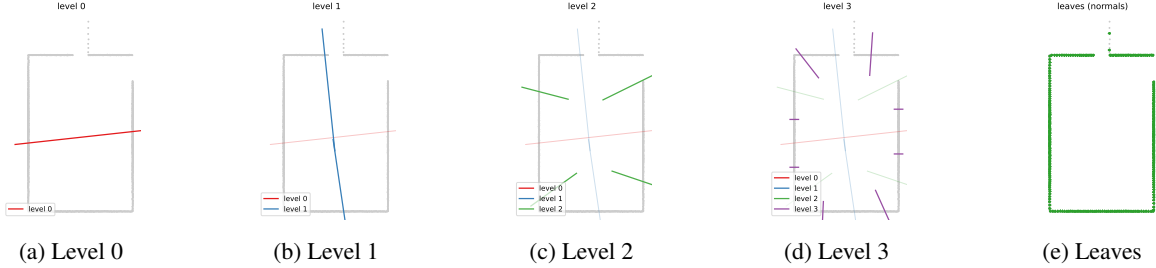


Figure 1: PCA KD-Tree construction on a synthetic 2D lidar scan of a room with a corridor visible through the doorway. Each panel adds one level; split planes are shown bold for the current level and faint for previous ones. The final panel shows leaf means with inward normals.

4 Key Properties of PCA KD-Trees

The data-driven construction of a PCA KD-Tree renders it equivariant with respect to isometric transformations. A node can be represented as a pair $\langle \mathbf{m}, \mathbf{d} \rangle$, where $\mathbf{m} \in \mathbb{R}^N$ is the mean of the points contributing to the node and $\mathbf{d} \in S^N$ is the direction unit vector. For intermediate nodes $\mathbf{d}$ is the largest eigenvector; for leaves it is the smallest eigenvector, encoding the surface normal.

Hence, applying an isometry $\mathbf{X} = [\mathbf{R} \mid \mathbf{t}]$ to a set of points P and then computing a PCA KD-Tree $\mathcal{T}$ is equivalent to computing a KD-Tree from P and then transforming each node as

$$\langle \mathbf{m}, \mathbf{d} \rangle \mapsto \langle \mathbf{X}\mathbf{m}, \mathbf{R}\mathbf{d} \rangle. \quad (1)$$

Additionally, assuming some ordering of the eigenvectors, the eigenvectors of the root node form a rotation matrix $\mathbf{R}_0 \in SO(N)$ and its mean gives a translation vector $\mathbf{t}_0$; together they form an isometry $\mathbf{X}_0 \in SE(N)$. Expressing all other nodes relative to this reference frame ensures that the root node lies at the origin while all other nodes are expressed relative to it. We call $\mathbf{X}_0$ the *root transform* of the tree.

Consider two point clouds P and P' , where P' is obtained by applying an isometry $\mathbf{X}$ to all elements of P , with corresponding KD-Trees $\mathcal{T}$ and $\mathcal{T}'$. Expressing all nodes of $\mathcal{T}$ and $\mathcal{T}'$ relative to their respective root transforms yields two identical structures. In short, PCA absorbs the relative transformation into the root node.

4.1 Deskewing via Tree Morphing

The equivariant property also helps with non-rigid transformations. In robotics it is common to work with moving sensors that produce a temporal stream of observations: every measurement is acquired at a slightly different sensor pose. Ignoring this effect leads to artefacts that are particularly evident when reconstructing 3D scenes with a fast-moving vehicle.

A common assumption is that the sensor moves with constant velocity $(\mathbf{v}, \boldsymbol{\omega})$ during the acquisition of one measurement frame. Given the timestamp t of each measurement, we compensate by expressing each local measurement $\mathbf{p}$ in the reference frame at the start of the stream t_0 :

$$\mathbf{p}' = \text{Exp}(\boldsymbol{\omega} \Delta t) \mathbf{p} + \mathbf{v} \Delta t, \quad \Delta t = t - t_0. \quad (2)$$

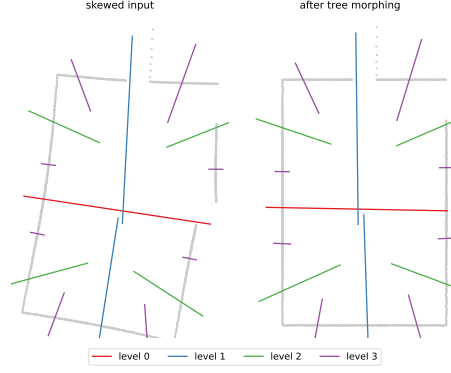


Figure 2: Effect of tree morphing on a skewed cloud. *Left*: KD-tree built directly on the skewed input; split planes follow the distorted geometry. *Right*: same tree after applying tree morphing with the sensor velocity; split planes recover the undistorted structure. Compare with Fig. 1 (level 3 panel) for reference.

The same mechanics can be applied to KD-Trees as a practical approximation. We store within each node the mean timestamp Δt of its points and use that value to morph the node. Let $\langle \mathbf{m}, \mathbf{d}, \Delta t \rangle$ be a node in the tree built from the original cloud; applying a constant-velocity model yields

$$\langle \mathbf{m}, \mathbf{d}, \Delta t \rangle \mapsto \langle \text{Exp}(\boldsymbol{\omega} \Delta t) \mathbf{m} + \mathbf{v} \Delta t, \text{Exp}(\boldsymbol{\omega} \Delta t) \mathbf{d}, 0 \rangle. \quad (3)$$

This is not an algebraic identity but a reasonable approximation. At zero velocity it is exact by construction. An example of the process is illustrated in Fig 2. For leaf nodes the temporal variance is small—a small spatial region corresponds to a small temporal slice for a uniformly sweeping sensor—so the approximation is tight there. For intermediate nodes the approximation is worse, since their temporal variance is larger. At the same time these morphed planes are still closer to the correct post-deskew geometry than their counterparts in the original tree. In the remainder of this work we present a series of experiments to assess the quality of this approximation.

5 KD-Trees for Registration

Registering two point clouds consists in finding an isometry $\mathbf{X}$ that, applied to a “moving” cloud, maximises its consistent overlap with a “fixed” cloud. When an initial guess is available, the most common approach is the ICP iterative scheme: at each round ICP refines $\mathbf{X}$ by nearest-neighbour search for correspondences and then updates $\mathbf{X}$ to minimise a distance metric between corresponding points.

ICP can also be applied to multiple clouds simultaneously, leading to the *multi-view registration* problem: find one transform per cloud that maximises consistent overlaps with all other clouds.

Remarkably, in PCA KD-Trees each level captures the scene at a different resolution. Each node has a specific geometric meaning: the vector $\mathbf{m}$ is the mean of the points in the node’s subcloud; for intermediate nodes $\mathbf{d}$ is the direction of maximum spread, while for leaves it is the surface normal. This multi-scale structure supports a recursive registration algorithm that matches each tree level and uses the result as initial guess for the next – someone should give it a try. The present work focuses on the leaves, which represent small surface patches. Let $\ell_a = \langle \mathbf{m}_a, \mathbf{d}_a \rangle$ and $\ell_b = \langle \mathbf{m}_b, \mathbf{d}_b \rangle$ be two matching leaves from clouds A and B , with cloud transforms $\mathbf{X}_a$ and $\mathbf{X}_b$. A convenient metric is

$$d(\ell_a, \mathbf{X}_a, \ell_b, \mathbf{X}_b) = (\mathbf{X}_a \mathbf{m}_a - \mathbf{X}_b \mathbf{m}_b) \cdot (\mathbf{R}_a \mathbf{d}_a + \mathbf{R}_b \mathbf{d}_b), \quad (4)$$

which projects the difference in means along the average normal. Guards on the angle between normals and on the point distance reject outliers. This metric is symmetric: the Jacobian $\mathbf{J}_b$ with respect to the perturbation $\delta \mathbf{x}_b$ is the negative of $\mathbf{J}_a$ with respect to $\delta \mathbf{x}_a$.

In a regular ICP scheme $\mathbf{X}_b$ is fixed and the minimisation affects only $\mathbf{X}_a$; in multiview registration both transforms are updated and the Jacobian symmetry yields faster computations. A single Gauss-Newton iteration is sketched in Alg. 4.

5.1 Continuous-Time ICP

The continuous-time (CT) ICP case is handled similarly, but the distance metric is augmented to include the sensor velocity $(\mathbf{v}_a, \boldsymbol{\omega}_a)$. The lookup point in the tree search becomes

$$\text{Exp}(\boldsymbol{\omega}_a \Delta t) \langle \mathbf{m}_a, \mathbf{d}_a \rangle, \quad (5)$$

Algorithm 4 Leaf-based ICP (rigid, one iteration)

```

1:  $\mathbf{H} \leftarrow \mathbf{0}, \mathbf{b} \leftarrow \mathbf{0}$ 
2:  $\mathbf{X}_\delta \leftarrow \mathbf{X}_b^{-1} \mathbf{X}_a$ 
3: for all  $\langle \mathbf{m}_a, \mathbf{d}_a \rangle \in \mathcal{T}_A$  do
4:    $\langle \mathbf{m}_b, \mathbf{d}_b \rangle \leftarrow \text{SEARCHTREE}(\mathcal{T}_B, \mathbf{X}_\delta \cdot \langle \mathbf{m}_a, \mathbf{d}_a \rangle)$ 
5:   if  $\neg \text{GOOD}(\langle \mathbf{m}_a, \mathbf{d}_a \rangle, \langle \mathbf{m}_b, \mathbf{d}_b \rangle)$  then
6:     continue
7:   end if
8:    $e \leftarrow (\mathbf{X}_a \mathbf{m}_a - \mathbf{X}_b \mathbf{m}_b) \cdot (\mathbf{R}_a \mathbf{d}_a + \mathbf{R}_b \mathbf{d}_b)$ 
9:    $\mathbf{J}_a \leftarrow \frac{\partial d(\ell_a, \text{Exp}(\delta \mathbf{x}_a) \mathbf{X}_a, \ell_b, \mathbf{X}_b)}{\partial \delta \mathbf{x}_a}$ 
10:   $\mathbf{H} \leftarrow \mathbf{H} + \mathbf{J}_a^\top \mathbf{J}_a, \mathbf{b} \leftarrow \mathbf{b} + \mathbf{J}_a^\top e$ 
11: end for
12:  $\delta \mathbf{x} \leftarrow \mathbf{H}^{-1}(-\mathbf{b})$ 
13:  $\mathbf{X}_a \leftarrow \text{Exp}(\delta \mathbf{x}) \mathbf{X}_a$ 

```

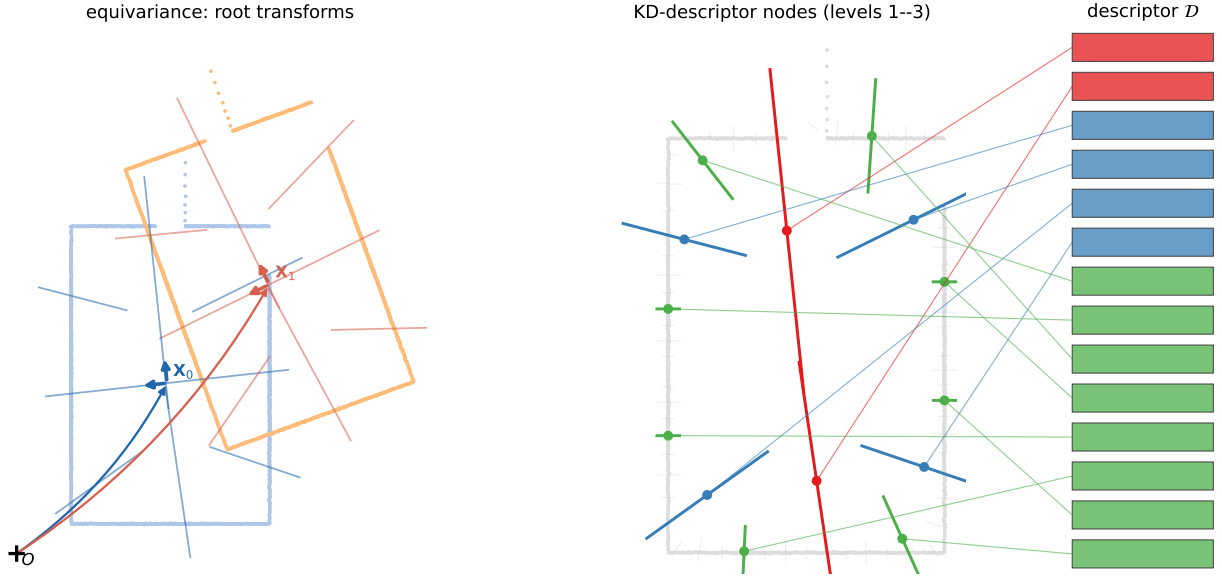


Figure 3: *Left*: the same scene observed from two different poses (blue and red). Each cloud’s root transform $\mathbf{X}_0, \mathbf{X}_1$ is shown as a pair of eigenvector axes; curved arrows indicate the displacement from the world origin O . *Center*: the first three BFS levels (highlighted) constitute the KD-Descriptor; node means are marked. *Right*: the descriptor vector $\mathcal{D}$ in BFS order, colored by level. A match between two descriptors immediately yields the relative transform $\mathbf{X}_0 \mathbf{X}_1^{-1}$ as an ICP seed.

and the velocities appear in the distance metric, resulting in a larger Jacobian and linear system. The solution update also covers the velocities:

$$[\delta \mathbf{x}, \delta \mathbf{v}, \delta \boldsymbol{\omega}] = \mathbf{H}^{-1}(-\mathbf{b}), \quad \mathbf{v}_a += \delta \mathbf{v}, \quad \boldsymbol{\omega}_a += \delta \boldsymbol{\omega}. \quad (6)$$

The Jacobians of multiview CT-ICP do not have the same symmetry as in the rigid case, since the velocity unknowns of A and B are in general different and do not compose symmetrically. However, several terms can be cached and shared between blocks of $\mathbf{H}$.

Remarkably, when using KD-Trees as the underlying primitive we can morph the trees based on the current velocity estimate at each iteration inexpensively, as described in Sec. 4.1.

6 KD-Trees for Place Recognition

In Sec. 4 we highlighted a powerful invariant of PCA KD-Trees: when nodes are expressed relative to the root transform, their values become invariant with respect to the global transformation of the cloud, which is entirely absorbed into the root transform.

This means that with a canonical assignment of eigenvector orientation, the intermediate nodes of a KD-Tree and its transformed counterpart will be identical. Hence the whole tree could serve as a holistic scene descriptor, comparable against a database using Euclidean distance. Such a metric would, however, be too tight for loop closure due to occlusions and viewpoint changes. The nodes close to the root capture the scene at a coarse level and respond with mild variations to small scene changes, whereas nodes closer to the leaves are more sensitive since they are built on a smaller statistic.

We therefore build a descriptor by stacking the first few levels (three in our experiments) of tree nodes into a vector. If two such vectors (KD-Descriptors) are close in Euclidean space, the corresponding clouds are likely similar. The root node always has zero mean after root-transform normalisation and does not appear in the KD-Descriptor; consequently a KD-Descriptor always has an even number of blocks.

Given two trees $\mathcal{T}_a$ and $\mathcal{T}_b$ whose descriptors match, a candidate relative transform is immediately available as $\mathbf{X}_a \mathbf{X}_b^{-1}$, which can be used to initialise ICP even without a prior estimate. In the limit case where the two clouds are geometrically identical, the descriptor match is exact and this transform is the true relative pose – no ICP required.

6.1 Canonization

Eigenvalue decomposition does not fix the orientation of eigenvectors; only their order (by magnitude) is determined. For an N -dimensional SPD matrix this leaves 2^{N-1} possible sign assignments, since the direction of the last eigenvector can always be chosen to make the eigenvector matrix right-handed. We call each such assignment a *canonization*.

Given the root transform we can always lift a KD-Descriptor to the node values computed before root normalisation, recompute the root transform under a new canonization, and recompute the descriptor. Hence a database stores only one descriptor per cloud. In the search phase we must run 2^{N-1} queries, one per canonization of the query descriptor.

A canonization is conveniently represented as a bit string of length $N - 1$: bit i is 1 if the i -th eigenvector axis is flipped with respect to the decomposition, and 0 otherwise. Alg. 5 shows how to obtain the alternative root rotation matrix $\mathbf{R}_{\text{new}}$ for a given canonization index.

Applying a canonization to a descriptor means re-expressing each block in the new root frame. A block $\langle \mathbf{m}, \mathbf{d} \rangle$ currently expressed relative to $\mathbf{X}_{\text{root}}$ becomes

$$\langle \mathbf{X}_{\text{new}}^{-1} \mathbf{X}_{\text{root}} \mathbf{m}, \mathbf{R}_{\text{new}}^{-1} \mathbf{R}_{\text{root}} \mathbf{d} \rangle. \quad (7)$$

Since the two root transforms share the same mean and differ only in the sign assignment of the eigenvectors, the product $\mathbf{R}_{\text{new}}^{-1} \mathbf{R}_{\text{root}}$ is a diagonal matrix with entries ± 1 , making this remapping inexpensive.

PCA is also used to determine the split plane of all intermediate nodes, introducing potential ambiguities in the plane normal orientation that result in different left/right node assignments in the tree. Two descriptors from the same cloud may therefore have different orderings of node entries.

6.2 MED-Compare

To cope with this artefact we replace Euclidean distance with an adaptive comparison. Instead of comparing nodes at fixed positions, we reshuffle nodes as the comparison progresses. Descriptor elements are laid out by BFS visit: the element $\mathcal{D}[2i]$ at position $2i$ represents a left node, and $\mathcal{D}[2i + 1]$ is the value of its right sibling. Given two descriptors $\mathcal{D}_a$ and $\mathcal{D}_b$, we compare sibling pairs at indices i and j by evaluating two hypotheses:

$$D_{\text{plain}} = \|\mathcal{D}_a[i] - \mathcal{D}_b[j]\| + \|\mathcal{D}_a[i+1] - \mathcal{D}_b[j+1]\|, \quad (8)$$

$$D_{\text{reverse}} = \|\mathcal{D}_a[i] - \mathcal{D}_b[j+1]\| + \|\mathcal{D}_a[i+1] - \mathcal{D}_b[j]\|, \quad (9)$$

where D_{plain} is the direct pairing and D_{reverse} is the distance obtained by swapping the left and right subtrees of b . We select the pairing that yields the smaller distance and recurse accordingly; the total distance is the sum over all sibling pairs.

Using integer arithmetic the comparison runs in time proportional to the descriptor length. We call this algorithm *MED-Compare* (Alg. 6). This is a heuristic that may suffer under symmetric configurations, but outperforms plain Euclidean comparison in our experiments.

Algorithm 5 Apply Canonization to Root Eigenvector Matrix

Require: eigenvector matrix $\mathbf{R} \in SO(N)$ (columns are eigenvectors), canonization index $c \in \{0, \dots, 2^{N-1} - 1\}$
Ensure: right-handed eigenvector matrix $\mathbf{R}'$ with flipped columns

```

1: function APPLYCANONIZATION( $\mathbf{R}, c$ )
2:    $\mathbf{R}' \leftarrow \mathbf{R}, f \leftarrow \text{false}$ 
3:   for  $i = 0, \dots, N - 2$  do
4:     if  $\text{bit}_i(c)$  then
5:        $\mathbf{R}'_{:,i} \leftarrow -\mathbf{R}'_{:,i}$ 
6:        $f \leftarrow \neg f$ 
7:     end if
8:   end for
9:   if  $f$  then ▷ restore  $\det(\mathbf{R}') = +1$ 
10:     $\mathbf{R}'_{:,N-1} \leftarrow -\mathbf{R}'_{:,N-1}$ 
11:   end if
12:   return  $\mathbf{R}'$ 
13: end function

```

Algorithm 6 MED-Compare: Mirroring-Equivariant Descriptor Comparison

Require: descriptors $\mathcal{D}_a, \mathcal{D}_b$ (BFS order, pairs at $2i, 2i+1$), current pair indices i_a, i_b , remaining pairs N , direction weight w
Ensure: distance $d \in \mathbb{R}$; initial call: MED-COMPARE($\mathcal{D}_a, \mathcal{D}_b, 0, 0, |\mathcal{D}|/2, w$)

```

1: function MED-COMPARE( $\mathcal{D}_a, \mathcal{D}_b, i_a, i_b, N, w$ )
2:   if  $N = 0$  then return 0
3:   end if
4:    $D_{\text{plain}} \leftarrow \|\mathcal{D}_a[i_a] - \mathcal{D}_b[i_b]\|_w + \|\mathcal{D}_a[i_a+1] - \mathcal{D}_b[i_b+1]\|_w$ 
5:    $D_{\text{reverse}} \leftarrow \|\mathcal{D}_a[i_a] - \mathcal{D}_b[i_b+1]\|_w + \|\mathcal{D}_a[i_a+1] - \mathcal{D}_b[i_b]\|_w$ 
6:   if  $D_{\text{plain}} \leq D_{\text{reverse}}$  then ▷ straight:  $a_l \leftrightarrow b_l, a_r \leftrightarrow b_r$ 
7:      $d_l \leftarrow \text{MED-COMPARE}(\mathcal{D}_a, \mathcal{D}_b, (i_a+1) \cdot 2, (i_b+1) \cdot 2, N/2, w)$ 
8:      $d_r \leftarrow \text{MED-COMPARE}(\mathcal{D}_a, \mathcal{D}_b, (i_a+2) \cdot 2, (i_b+2) \cdot 2, N/2, w)$ 
9:     return  $D_{\text{plain}} + d_l + d_r$ 
10:  else ▷ swapped:  $a_l \leftrightarrow b_r, a_r \leftrightarrow b_l$ 
11:     $d_l \leftarrow \text{MED-COMPARE}(\mathcal{D}_a, \mathcal{D}_b, (i_a+1) \cdot 2, (i_b+2) \cdot 2, N/2, w)$ 
12:     $d_r \leftarrow \text{MED-COMPARE}(\mathcal{D}_a, \mathcal{D}_b, (i_a+2) \cdot 2, (i_b+1) \cdot 2, N/2, w)$ 
13:    return  $D_{\text{reverse}} + d_l + d_r$ 
14:  end if
15: end function

```

7 Building on the Blocks: KD-SLAM

In this section we assemble a SLAM system that uses KD-Trees as a universal geometric primitive. The original goal was to provide a tool that is elegant, modular, and requires limited resources. It turned out to give promising results without domain-specific tuning.

KD-SLAM implements a classical pose-graph scheme. The map is a pose-graph whose nodes carry keyframes, each being a KD-Tree. Since the leaves of the tree preserve sufficient geometric information, the original point cloud is discarded after tree construction.

A pose-graph is a special case of a factor graph where each node stores the sensor pose at which a keyframe was acquired. Edges encode relative pose measurements from cloud registration; the information matrix of each edge is a by-product of the registration process.

The overall data flow is illustrated in Fig. 5.

(a) KD-Tree construction. Once a point cloud becomes available the KD-Tree is constructed on a separate I/O thread. Pre-computed trees can also be loaded from disk, which is particularly convenient for large-scale experiments: the overall system turns out to be bound by the bag reading pipeline; pre-computing trees achieves data compression that lowers that pressure.

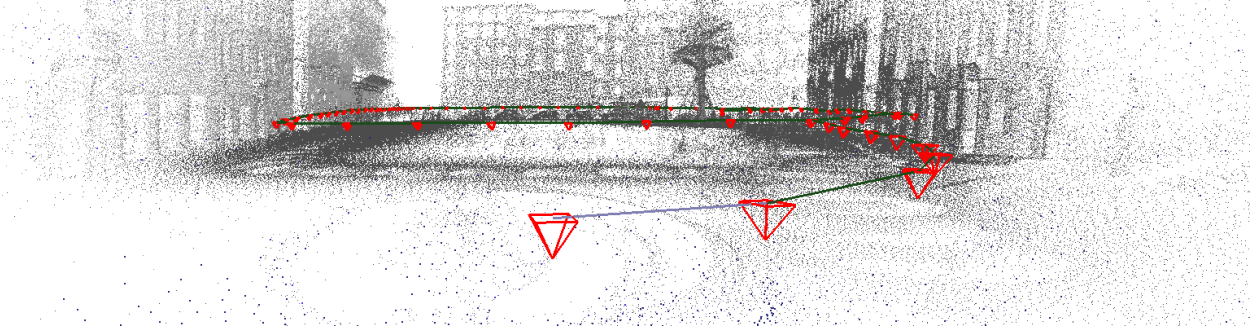


Figure 4: KD-SLAM running on a large handheld. Keyframes (red) and pose-graph edges are visible in the viewer. The floating frame is in foreground, connected to its keyframe by a gray edge.

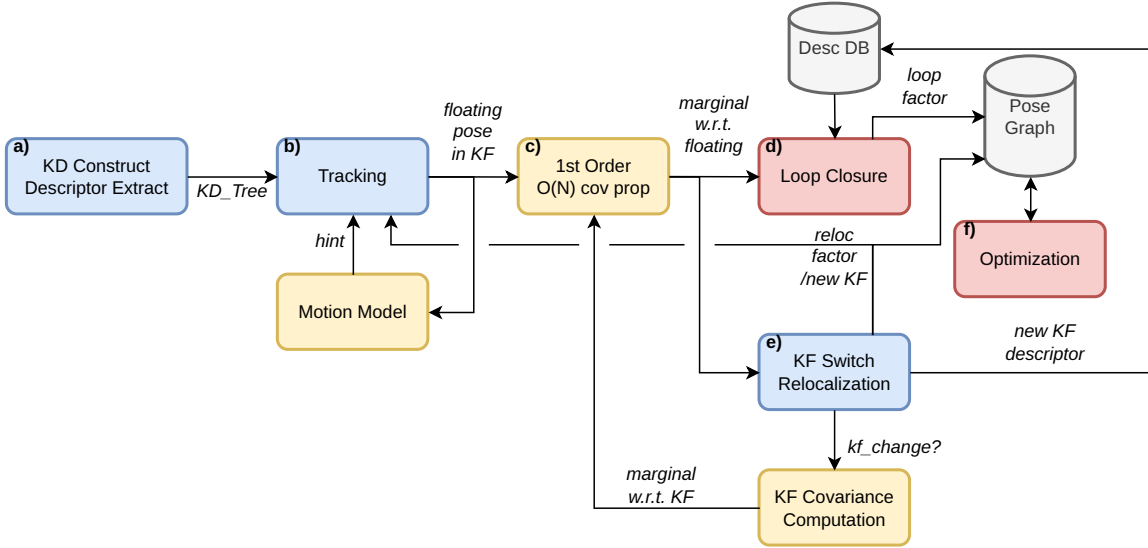


Figure 5: Block diagram of the KD-SLAM pipeline. Arrows indicate data flow between the main processing stages: tree construction, tracking, covariance propagation, loop detection, keyframe management, and pose-graph optimisation.

(b) Tracking. The pose of the floating frame is updated with a motion model. Currently we support IMU integration, wheel odometry, constant velocity, and identity. The floating frame is registered against the current keyframe; if a velocity estimate is available (as in CT-ICP) the velocities are applied to the floating frame and the registration output is fed back to the motion model.

(c) Marginal Covariances. For the subsequent stages we need to compute the marginal covariances of all keyframes w.r.t. the current node. To achieve this in $O(\text{keyframes})$ we rely on the marginals of all nodes w.r.t. the current keyframe that are computed whenever the keyframe switches, and we simply compound the covariance between the keyframe and the floating frame with all keyframe marginals.

(d) Loop detection. To find potential loop closures we combine covariance-based keyframe selection with descriptor matching. We select all keyframes whose positional offset from the current keyframe passes a Mahalanobis gate on the keyframe marginal covariance, and whose graph distance exceeds a threshold. A small slack is applied before the test to account for small displacements.

We compare the current descriptor (with all its canonizations) against the surviving keyframes using MED-Compare. The descriptor match provides a candidate transform $\mathbf{X}_{\text{loop},c}$ between the floating frame and the matching keyframe – the initial guess for ICP comes directly from the descriptors, so loop closures can be recovered from arbitrarily large positional offsets. From incremental tracking we also have an estimate $\mathbf{X}_{c,kf}$. To further prune candidates we apply a zero-shot consensus check: let $\mathbf{X}_{kf,\text{loop}}$ be the transform from matching the loop and current keyframe descriptors; we

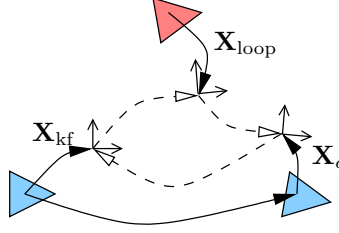


Figure 6: Illustration of the zero-shot consensus check. Robot poses are depicted as triangles, each with its KD-tree root transform attached. Tracking provides the relative transform between the floating frame and the keyframe, giving the relative position of $\mathbf{X}_{kf}$ and $\mathbf{X}_c$. Descriptor matching yields the transform between the floating frame and the loop candidate ($\mathbf{X}_{loop,c}$) and between the keyframe and the loop candidate ($\mathbf{X}_{kf,loop}$). If the kinematic chain does not close, the descriptor matches disagree with tracking and the candidate is rejected.

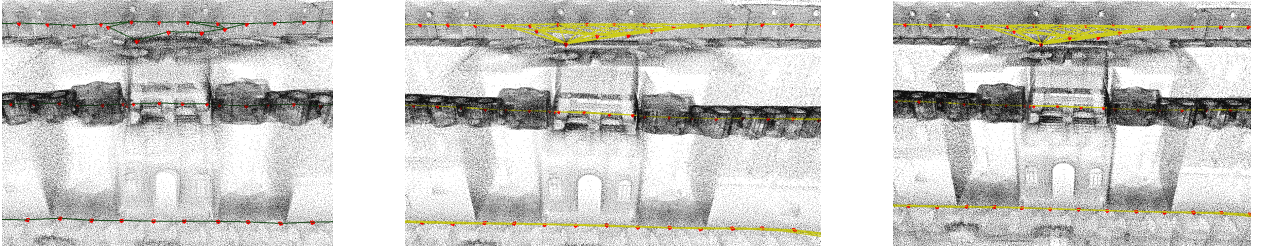


Figure 7: Effects of Multi-View. Left: a map artificially perturbed by injecting errors in the poses and in the keyframe velocities. Middle: after multiview rigid registration, most of the structure is recovered. Right: after CT registration on the rigid multiview output: estimated DoF of velocity eliminates some artifact.

accept the candidate only if

$$\mathbf{X}_{loop,c} \cdot \mathbf{X}_{c,kf} \cdot \mathbf{X}_{kf,loop} \approx \mathbf{I}. \quad (10)$$

Candidates that pass this kinematic chain check (stage 2, loop consensus) proceed to stage 3: the orientation of the proposed factor is compared against the delta orientation accumulated in the PGO; candidates with large discrepancies are rejected before ICP is run.

The idea behind the loop consensus is illustrated in Fig. 6. The ICP trials are validated by ICP and, if the overlap is sufficient, turned into PGO edges.

(e) Keyframe switching and relocalization. If the overlap of the floating frame with its keyframe falls below a threshold, instead of immediately spawning a new keyframe we first search for an existing keyframe in the neighbourhood that can provide sufficient support. We apply the same χ^2 test used in loop detection, restricting candidates to keyframes close in the graph. We register the floating frame against each candidate and adopt the best match as the new current keyframe, reparenting the floating frame to it. If relocalization fails, the floating frame becomes a new keyframe and the tracking ICP edge is added to the PGO.

(f) Optimization. Each time a new edge is added to the PGO we monitor the χ^2 increase. Open-loop and relocalization edges typically contribute little to χ^2 , so a global optimization is triggered only when the accumulated increase exceeds a threshold. Loop closures generally trigger an immediate optimization; several relocalization events may be accumulated before triggering one.

Marginal covariances. The marginal covariance of all keyframes with respect to the current keyframe is computed via BFS using first-order error propagation, and runs only when the PGO topology changes. The covariance between the floating frame and all other keyframes is obtained in time linear in the number of nodes by chaining the transform uncertainty between the floating frame and its keyframe with the BFS-propagated covariance.

8 Building on the Built Blocks: KD-Bundle

A design suggestion for SLAM systems: favor topological consistency over metric accuracy in the online phase. Once the topology is right, metric accuracy can be tightened offline by running multi-view ICP over the keyframe graph.

The idea is straightforward: take the poses and keyframes from the PGO, build a co-visibility graph, and cast a multi-view ICP factor between every pair of keyframes that see each other. To efficiently compute the co-visibility we store the global translations of the keyframes in a KD-tree and we run a complete nearest neighbor search. We start with loose bounds, and we deactivate factors whose number of inliers is too low as the computation proceeds. The active factors define the sparsity pattern of the linear system: the resulting least-squares system has a natural block-sparse structure — each keyframe pose occupies a 6×6 diagonal block (or 12×12 when velocities are in the state), with an off-diagonal block at position (i, j) whenever frames i and j share a factor. We inject Hessian blocks directly into our own solver [19], since it was designed to support this feature.

The per-factor contributions to $\mathbf{H}$ and $\mathbf{b}$ are independent and embarrassingly parallel (Alg. 7; notation as in Sec. 5). In the CT-ICP variant, each iteration opens with a backup of the current leaf positions and a fresh deskew using the current velocity estimate; after the perturbation is applied the trees are restored to their undeskewed state, ready for the next backup. The leaves themselves are never touched during deskewing — only the search nodes are moved — so Jacobians are always evaluated against the original geometry.

In the rigid case $\mathbf{J}_b = -\mathbf{J}_a$ exactly, hence $\mathbf{H}_{aa} = \mathbf{H}_{bb}$ and $\mathbf{H}_{ab} = -\mathbf{H}_{ba}$: only $\mathbf{H}_{aa}$ and $\mathbf{H}_{ab}$ need to be computed, halving the work per factor. Together with the drop from 12×12 to 6×6 blocks, the rigid variant costs less than one-sixteenth of the full CT case.

Algorithm 7 Multi-view ICP — one iteration

```

1: if CT-ICP then
2:   for all keyframe trees  $\mathcal{T}_k$  do
3:     backup( $\mathcal{T}_k$ )
4:     deskew( $\mathcal{T}_k, \dot{\mathbf{X}}_k$ ) ▷ search nodes only; leaves unchanged
5:   end for
6: end if
7:  $\mathbf{H} \leftarrow \mathbf{0}, \mathbf{b} \leftarrow \mathbf{0}$ 
8: for all factor  $(\mathcal{T}_A, \mathcal{T}_B) \in \mathcal{F}$  do ▷ parallel
9:    $\mathbf{H}_{aa}, \mathbf{H}_{ab}, \mathbf{H}_{bb}, \mathbf{b}_a, \mathbf{b}_b \leftarrow \mathbf{0}$ 
10:   $\mathbf{X}_\delta \leftarrow \mathbf{X}_B^{-1} \mathbf{X}_A$ 
11:  for all leaf  $\langle \mathbf{m}_a, \mathbf{d}_a \rangle \in \mathcal{T}_A$  do
12:     $\langle \mathbf{m}_b, \mathbf{d}_b \rangle \leftarrow \text{SEARCH}(\mathcal{T}_B, \mathbf{X}_\delta \cdot \langle \mathbf{m}_a, \mathbf{d}_a \rangle)$ 
13:    if  $\neg \text{GOOD}(\langle \mathbf{m}_a, \mathbf{d}_a \rangle, \langle \mathbf{m}_b, \mathbf{d}_b \rangle)$  then
14:      continue
15:    end if
16:     $\mathbf{n} \leftarrow \frac{1}{2}(\mathbf{R}_A \mathbf{d}_a + \mathbf{R}_B \mathbf{d}_b)$ 
17:     $\mathbf{e} \leftarrow (\mathbf{X}_A \mathbf{m}_a - \mathbf{X}_B \mathbf{m}_b) \cdot \mathbf{n}$ 
18:     $\mathbf{J}_a \leftarrow \partial e / \partial \delta \mathbf{x}_a|_0, \mathbf{J}_b \leftarrow \partial e / \partial \delta \mathbf{x}_b|_0$ 
19:     $\mathbf{H}_{aa} += \mathbf{J}_a^\top \mathbf{J}_a, \mathbf{b}_a += \mathbf{J}_a^\top \mathbf{e}$ 
20:     $\mathbf{H}_{bb} += \mathbf{J}_b^\top \mathbf{J}_b, \mathbf{b}_b += \mathbf{J}_b^\top \mathbf{e}$ 
21:     $\mathbf{H}_{ab} += \mathbf{J}_a^\top \mathbf{J}_b$ 
22:  end for
23:   $\mathbf{H}[A, A] += \mathbf{H}_{aa}, \mathbf{H}[A, B] += \mathbf{H}_{ab}, \mathbf{H}[B, B] += \mathbf{H}_{bb}$ 
24:   $\mathbf{b}[A] += \mathbf{b}_a, \mathbf{b}[B] += \mathbf{b}_b$ 
25: end for
26:  $\delta \mathbf{x} \leftarrow \text{SOLVE}(\mathbf{H}, \mathbf{b})$ 
27: for all keyframe  $k$  do
28:    $\mathbf{X}_k \leftarrow \text{Exp}(\delta \mathbf{x}_k) \mathbf{X}_k$ 
29: end for
30: if CT-ICP then
31:   for all keyframe trees  $\mathcal{T}_k$  do
32:     restore( $\mathcal{T}_k$ ) ▷ undo deskew for next iteration
33:   end for
34: end if

```

9 Implementation

The algorithms described in the previous sections admit a particularly clean implementation that avoids dynamic memory allocation and is friendly to modern cache hierarchies.

The whole infrastructure is dimension and operation agnostic, instantiated by providing a traits class that defines the scalar type, the dimensions, and the domain operations (expmap, logmap, transformations). The 2D/3D switch is a single traits swap.

9.1 Tree

The tree nodes are stored in a flat array. Each node contains mean, direction, and indices to the left and right children. The flat structure allows seamless interaction between GPU and CPU: the tree is a plain-old-data struct holding pointers to arrays, the canonical transformation, and some statistics.

In-place partitioning at node split At each node, the point set is split without copying. Two indices s and e start at the left and right ends of the current range; s advances when the point at position s lies on the correct side of the splitting plane, otherwise the points at s and e are swapped and e retreats. The loop terminates when $s > e$, having partitioned the subarray in-place in linear time and zero extra allocation.

BFS node ordering. Nodes are pre-allocated in a flat vector and filled in breadth-first order. For a node at index k , its left and right children reside at known offsets, so child pointers reduce to integer indices into the same vector. BFS ordering ensures that nodes visited consecutively during a top-down traversal are also contiguous in memory, maximising cache reuse.

Embarrassingly parallel construction. Once the root split is computed the two subtrees are fully independent. Construction is therefore parallelised by spawning a thread per subtree down to a user-defined depth, after which the recursive calls run serially. The node vector and the point array are pre-allocated before any thread is spawned, so there is no contention on the allocator.

9.2 Descriptors

The descriptors are stored as flat arrays whose elements represent the means and direction vectors of the corresponding nodes, indexed to match the keyframe they belong to. MED-Compare is implemented as an iterative scheme with minimal branching and runs on both CPU and GPU.

9.3 ICP

The ICP/CT-ICP engine converges in a few iterations, stopping when a termination criterion is met: the inlier count stabilises and χ^2 stops decreasing. The engine is the building block of the whole system; a single class supports both single-frame and dual-frame (multiview) operation. The CPU and CUDA instantiations derive from a common base and share the same core loop for tree search and Jacobian computation, further specialised by the domain traits.

9.4 Faster than the bag reader.

Converting the bag takes 12 minutes. Reading the bag takes 12 minutes. Running SLAM takes 45 seconds. The bottleneck is the bag reader, not the pipeline. The practical workflow is: convert once when sensor parameters are settled, run from trees thereafter. In an online deployment the bag reader is replaced by the LiDAR driver node; the pipeline headroom remains.

10 Experiments

We evaluate KD-SLAM as a study, not a competition. The goal is to understand what the PCA KD-tree buys at each stage of the pipeline – odometry, loop closure, multiview registration, continuous-time refinement – and where it does not yet pay off. Experiments run on two machines: a laptop (Intel Core i7-8750H, 6 cores, GTX 1060) used for development, and an Intel Pentium J4205 Apollo Lake (4 cores, 10 W, no GPU) representative of embedded deployment. One parameter configuration per sensor; no per-sequence tuning.

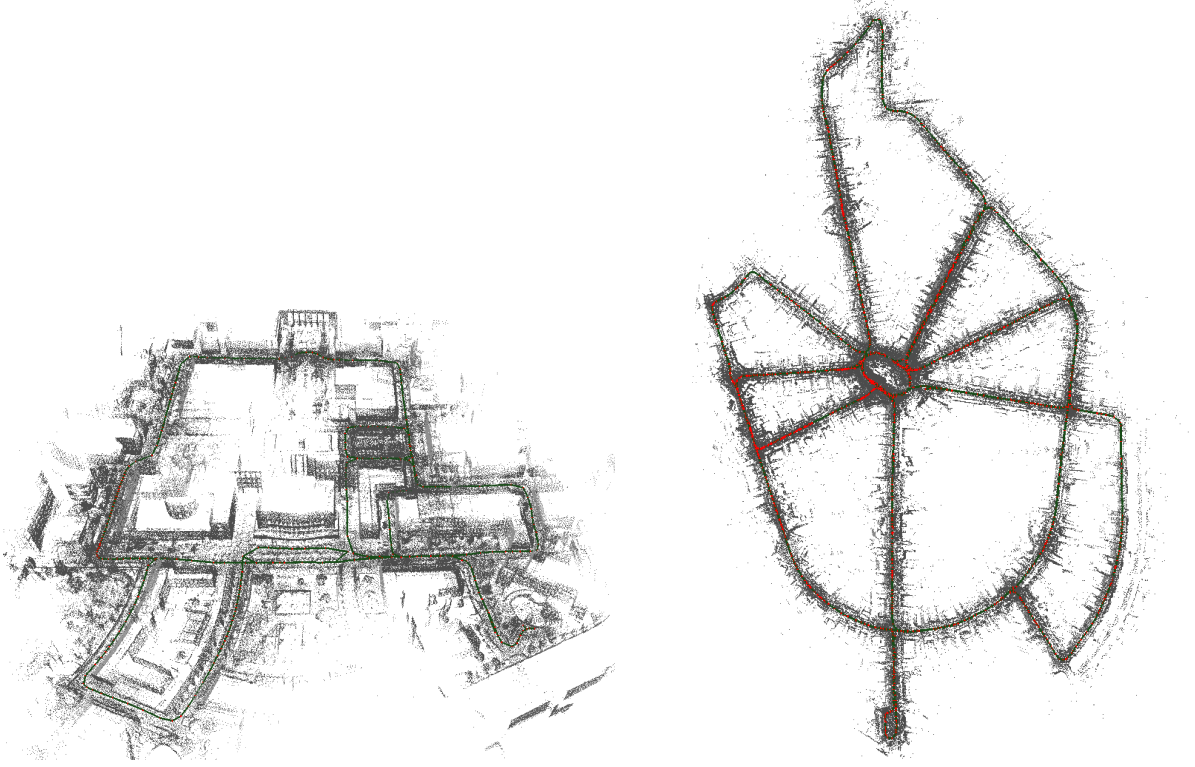


Figure 8: Maps built from the VBR sequences, drive configuration: campus (left) and ciampino (right).

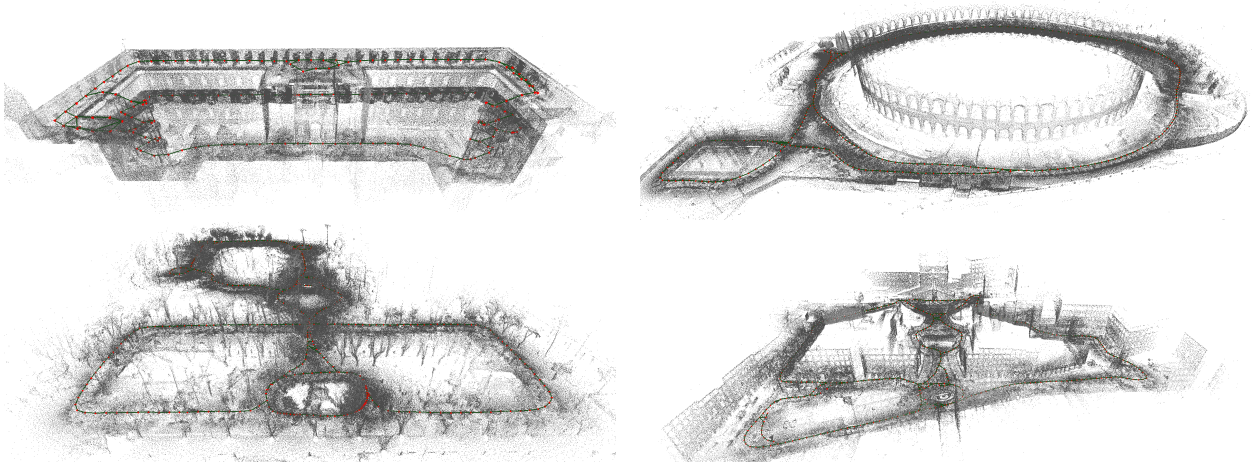


Figure 9: Maps built from the VBR sequences, handheld configuration. From top to bottom: diag, colosseo, pincio, spagna.

10.1 VBR Benchmark

We tested on the VBR benchmark [3](Brizi, Giacomini *et al.*) using the training split (ground truth available), scored with the VBR evaluation tools [3]. Sequences span two sensor types: Ouster OS0 for handheld and Ouster OS1 for drive. We report six variants: plain odometry (ICP, CT-ICP), plus multiview registration (MV) and a subsequent CT refinement pass (CT).

ICP and CT-ICP are comparable on VBR (RPE_T 0.69% vs 0.93%, ATE_T 0.87 m vs 0.88 m). CT-ICP is a harder matching problem: resolving the continuous-time term requires dense, stable support, which is why most methods

Table 1: VBR training sequences: loop closure funnel and timing. Funnel columns: spatial χ^2 gate (Sp), appearance filter (D1), loop consensus (D2), ICP trials (Tr), confirmed (OK). Timing with precomputed trees on i7 (CUDA) and J4205 Apollo Lake (CPU, 10 W).

Sequence	ICP							CT-ICP						
	spatial	D1	D2	ICP	OK	i7 ms	J4 ms	spatial	D1	D2	ICP	OK	i7 ms	J4 ms
diag	7415	3412	285	32	2	4.0	14.6	4340	2025	126	31	2	20.3	48.1
colosseo	1290	1224	154	108	2	3.8	12.6	952	865	98	56	2	18.5	39.5
pincio	4793	4792	650	222	9	4.2	14.0	3040	3040	414	105	6	19.2	40.9
spagna	1789	1030	136	77	3	4.4	13.4	987	682	68	40	3	16.7	41.5
ciampino-0	10966	8158	1636	264	24	4.2	11.3	9694	7388	1456	212	26	8.9	18.2
ciampino-1	3938	2858	238	60	13	3.0	7.8	4560	3212	220	57	13	6.3	13.1
campus-0	898	511	17	8	4	2.6	7.1	965	735	24	7	3	5.6	12.1
campus-1	724	530	15	15	4	2.9	6.9	771	541	10	10	4	5.5	11.7

Table 2: VBR training sequences: accuracy for ICP variants. e_t (%) = RPE_T , e_r (deg/m) = RPE_R , T (m) = ATE_T , R (deg) = ATE_R .

Sequence	ICP				ICP+MV				ICP+MV+CT			
	e_t	e_r	T	R	e_t	e_r	T	R	e_t	e_r	T	R
diag	0.59	0.029	0.42	0.99	0.52	0.027	0.35	0.85	0.52	0.027	0.35	0.85
colosseo	0.52	0.014	0.58	0.81	0.60	0.015	0.62	0.88	0.60	0.015	0.62	0.88
pincio	0.49	0.013	0.69	0.88	0.45	0.014	0.53	0.85	0.45	0.014	0.53	0.85
spagna	0.35	0.010	0.45	0.69	0.34	0.010	0.36	0.66	0.34	0.010	0.36	0.66
ciampino-0	1.51	0.008	1.97	1.71	1.41	0.008	1.99	1.68	1.41	0.008	1.99	1.68
ciampino-1	1.17	0.009	1.50	1.46	1.17	0.009	1.50	1.46	1.16	0.009	1.50	1.46
campus-0	0.46	0.007	0.67	0.88	0.44	0.007	0.69	0.85	0.44	0.007	0.69	0.85
campus-1	0.46	0.006	0.67	0.81	0.41	0.005	0.68	0.76	0.41	0.005	0.68	0.76
Avg	0.69	0.011	0.87	1.03	0.67	0.012	0.84	1.00	0.67	0.012	0.84	1.00

maintain a local accumulated map. We match against a single keyframe by design. MV consistently improves both ICP (RPE_T 0.69% $\rightarrow$ 0.67%, ATE_T 0.87 $\rightarrow$ 0.84 m) and CT-ICP (ATE_T 0.88 $\rightarrow$ 0.85 m). The CT refinement pass after MV changes nothing: CT+MV and CT+MV+CT are the same number, rounded. The quantitative improvement is marginal; the qualitative effect on map crispness is visible in Fig. 7.

Loop closure funnel. On handheld sequences the spatial χ^2 gate is tight: only geometrically close candidates pass, and those tend to be similar in appearance. D1 and D2 cost nothing; on these sequences they barely need to act (pincio: 4793 $\rightarrow$ 4792 after D1), and D2 handles the residual tail. On drive sequences the gate is looser – keyframes are sparser and drift larger – so more candidates survive the spatial test, and D1 carries the filtering load (ciampino-0: 10966 $\rightarrow$ 8158 $\rightarrow$ 1636 $\rightarrow$ 264 $\rightarrow$ 24). Voxelization sharpens D1 further on handheld sequences (spagna D1 reduction: 42% $\rightarrow$ 55%).

Timing. Same vintage laptop, same config switch: ICP runs at 2.6–4.4 ms per frame, 20–40 $\times$ real time. On the Apollo Lake, 6.9–14.6 ms – still comfortable real time. CT-ICP costs up to 3 $\times$ more on the Apollo Lake (11.7–48.1 ms); on the i7 the gap is smaller (5.5–20.3 ms) because CUDA hides the extra computation. Handheld sequences are slower than drive: the OS0 produces denser clouds than the OS1, and the tree size drives everything.

10.2 KITTI Odometry Benchmark

We tested on all 11 sequences (00–10) of the KITTI odometry benchmark [15]. No IMU, no ground plane extraction. Clouds are pre-deskewed. Metrics are translation RPE (% of path length) and rotation RPE (deg/m), computed with the official KITTI devkit [15].

Sequences with loop closure (00, 05, 06, 07, 09) all land below 0.87%. Open-loop sequences (03, 04, 10) sit around 0.86–1.01%: smooth drift, no collapse. Odometry-tuned systems do better on open-loop sequences; closing that gap would require a local accumulated map and more – neither of which is in scope. That the system performs as it does with a single keyframe for support is the point.

Loop closure funnel. KITTI clouds are voxelized, which stabilizes PCA axes at coarse tree levels and sharpens D1 discrimination (Seq 00: 2729 $\rightarrow$ 931, 66% reduction). Sequence 02: 4803 spatial candidates, 58 ICP trials after the appearance and consensus stages – 80 $\times$ reduction. Eight confirm, all outside the tunnel. Sequence 08: 4001 spatial

Table 3: VBR training sequences: accuracy for CT-ICP variants. e_t (%) = RPE_T , e_r (deg/m) = RPE_R , T (m) = ATE_T , R (deg) = ATE_R .

Sequence	CT-ICP				CT+MV				CT+MV+CT			
	e_t	e_r	T	R	e_t	e_r	T	R	e_t	e_r	T	R
diag	0.86	0.046	0.39	1.15	0.83	0.046	0.37	1.09	0.83	0.046	0.37	1.09
colosseo	0.96	0.022	0.52	0.94	1.07	0.024	0.56	1.11	1.07	0.024	0.56	1.11
pincio	1.13	0.034	0.62	1.21	1.05	0.031	0.42	1.10	1.05	0.031	0.42	1.10
spagna	0.64	0.020	0.43	0.92	0.66	0.020	0.42	0.92	0.66	0.020	0.42	0.92
ciampino-0	1.57	0.009	2.13	1.79	1.52	0.009	2.11	1.78	1.52	0.009	2.11	1.78
ciampino-1	1.29	0.011	1.60	1.60	1.27	0.011	1.65	1.64	1.27	0.011	1.65	1.64
campus-0	0.52	0.009	0.73	0.90	0.49	0.008	0.66	0.84	0.49	0.008	0.66	0.84
campus-1	0.47	0.007	0.60	0.79	0.47	0.007	0.60	0.79	0.47	0.007	0.60	0.79
Avg	0.93	0.020	0.88	1.16	0.92	0.019	0.85	1.16	0.92	0.019	0.85	1.16

Table 4: KITTI odometry: accuracy, loop closure funnel, and timing. Funnel columns: spatial χ^2 gate (spatial), appearance filter (D1), loop consensus (D2), ICP trials (ICP), confirmed (OK). Timing measured with precomputed trees on i7 (CUDA) and Intel J4205 Apollo Lake (CPU, 10 W).

Seq	Len (m)	e_t (%)	e_r (deg/m)	Loop closure funnel					i7 (CUDA)		J4205 (CPU)	
				spatial	D1	D2	ICP	OK	ms/f	$\times$ RT	ms/f	$\times$ RT
00	3724	0.82	0.00347	2729	931	656	74	15	3.7	27	6.4	16
01	2453	1.95	0.00413	0	0	0	0	0	5.5	18	10.8	9
02	5067	1.43	0.00600	4803	2742	2534	58	8	3.6	28	6.9	14
03	561	1.01	0.00554	0	0	0	0	0	4.7	21	6.7	15
04	394	0.86	0.00737	0	0	0	0	0	8.3	12	8.7	12
05	2206	0.41	0.00214	1571	467	311	81	26	3.4	30	6.6	15
06	1233	0.59	0.00254	134	128	117	23	3	4.0	25	7.0	14
07	695	0.62	0.00472	287	140	139	1	1	3.8	26	5.6	18
08	3223	1.07	0.00428	4001	1436	1387	86	2	3.9	26	9.2	11
09	1705	0.86	0.00311	879	49	49	31	3	4.7	21	7.9	13
10	920	0.99	0.00509	0	0	0	0	0	4.1	24	5.9	17
Avg		1.01	0.00440						3.9	23	7.4	14

candidates, 86 trials, 2 confirm. Both are correct and needed. Once relocalized, the spatial χ^2 gate rejects all subsequent candidates – they lie within already-mapped territory. Sequence 07: one trial, one confirmation. Clean loop, clean result.

Timing. Numbers were collected on a vintage laptop (i7 + GTX 1060) used while writing this paper. It works well: 3.4–8.3 ms per frame, 12–30 $\times$ real time at 10 Hz. The same binary, config change only, runs on a 10 W Apollo Lake J4205 at 5.6–10.8 ms per frame. Sequence 01 is the J4205 outlier: faster motion means a larger frame-to-frame offset, and ICP needs more iterations to converge. If it runs on an Apollo Lake it very likely runs on a single slow core of a modern smartphone. ICP is embarrassingly parallel. We added multithreading; on this memory-bound machine the gain is modest.

Test sequences 11–21. We processed all eleven test sequences with the same binary and configuration used for the training set. The results have been submitted to the KITTI evaluation server. At the time of this writing the leaderboard numbers are not yet available; the reconstructed maps are geometrically consistent and visually indistinguishable in character from those of the training sequences. Fig. 11 illustrates the maps recovered from some loopy sequences.

11 Conclusions

The PCA KD-tree carries further than expected. A single structure, built once per scan, supports deskewing, odometry, place recognition, and multi-view registration without switching representation – and does so in real time on hardware most robots already carry.

Two things do not yet pay off. CT-ICP on a single keyframe provides poor support for the continuous-time term; a local accumulated map would help, and is the natural next step. Pitch drift on drive datasets points to a range-dependent elevation warp in the sensor model that we have not yet characterized.

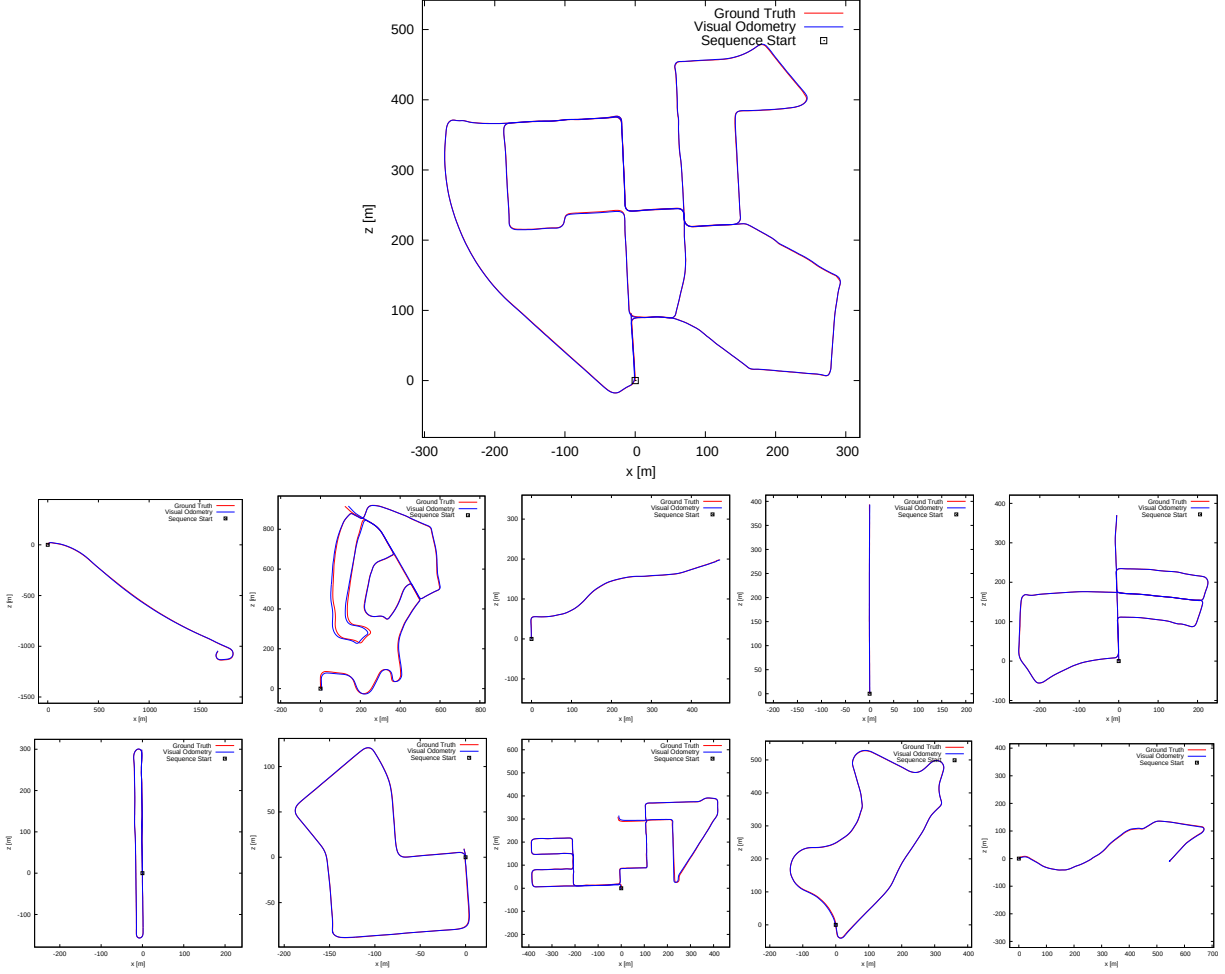


Figure 10: KITTİ odometry sequences 00–10. Ground truth (red) vs. KD-SLAM estimate (blue), Umeyama-aligned. Best viewed in color.

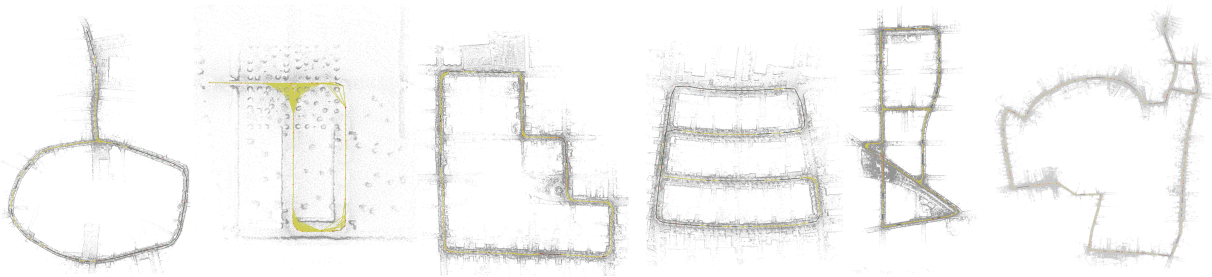


Figure 11: Some loopy sequences (13, 14, 15, 16, 18 and 19) of the KITTİ test dataset. The maps look visually consistent.

Those who cannot refrain from further thought have our sympathy. The eigenvalues already encode geometric saliency; gating keyframe admission on them rather than accepting every scan is obvious enough that it is embarrassing to write down. Nevertheless, IMU pre-integration slots in as a motion model without disturbing the rest of the pipeline. Straightforward. An offline `kd_map_doctor` that re-seeks loop closures aggressively on the finished map, before multiview registration, would recover the topological gaps that real-time conservatism is forced to leave open. Worth a Master's thesis. KD-trees do not compose naively, but a patient sitting-and-thinking session might yield an algorithm that amortizes the merge cost and opens the door to local maps. Someone should think about this on a Sunday. Most

dangerously: the split plane is the only geometric locus where two adjacent leaves share a boundary; chasing neighbors exclusively through split planes is a surface reconstruction algorithm hiding inside the tree. Do not.

Should someone be willing to chase the benchmarks starting from this infrastructure, we would recommend: IMU pre-integration as a motion model, keyframe selection driven by eigenvalue diversity, and an offline loop-closure pass before multiview registration. Good luck.

A KD-SLAM: the story behind

Revision 1 of KD-SLAM was ready in the first days of July 2024, as a result of three intensive months of solo work on CUDA and parallel architectures. In that revision I was playing with parallel algorithms and I was thrilled to have a system that runs mostly on GPU. I spent considerable effort in the construction of the tree in GPU, that worked but did not present evident advantages w.r.t. its CPU counterpart. KD-SLAM 1 was essentially a 3D rigid pipeline with the multiview refinement of what you read in this work. Then I was happy with the results and, as I sometimes do, lost interest in sharing the results with the community. Two things got in the way. Looking at the publications and the trends in the computer vision and robotics communities it appears to me that to be well received a work needs to have some LLM/Deep neural network component, and should solve some problem that is somehow hard to model. This is particularly true in the young audience. As a professor and a teacher when I write a paper, I do it primarily for the students. If they are not interested, the paper does not need to exist. When I told my students “look we have a decent Lidar SLAM system”, they looked at me with a bored face and answered “another one?”.

KD-SLAM lies at the opposite of this domain: it is a simple, explainable old-school well engineered system that builds on squeezing a data structure and its properties to the limit. I find it elegant, compact and, should you ever look at the code, a rather good piece of software. With the *same* C++ template plumbing KD-SLAM supports in a composable manner 2D/3D pipelines, it handles rigid and continuous time cases, and it can get deployed to CPU only systems or CPU+GPU.

Another reason of the delay was my positive response to a call of duty, and I was appointed Director of a National PhD program in AI. As cool as it sounds the role consists in managing 120 students, distributed across the country managing infinite paperwork dictated by stratified layers of bureaucracy that turned me into an email router dealing with information systems that break every possible usability guideline and potentially introduce new ones. This task absorbed basically all time I devoted to research and “playing”. Clearly, this extra task came without any form of compensation.

Covering this role provoked in me a repulsion for anything that was remotely connected to AI. My students were delving into GPT/Claude as coding assistants, while I stayed out. Some years before I tried these tools, but as a vintage emacs/linux/c++ developer I was becoming irritated every time the system was proposing a suggestion. This was disrupting my logic flow.

The last spark of coding was in late 2024, when I stripped the ICP part from KD-SLAM and used it as a teaching material, generalizing over 2D/3D. It was then that I introduced traits to nail down code replication and describe an algorithm as built on domain primitives enclosed in traits.

Needless to say that after I got appointed, I entirely stopped coding. It was early 2025, and I was even reconsidering my role in academia. I did not choose this path to become a bureaucrat to solve problems that should not exist in the first place. As a teacher, if the students are not interested in what I have to tell, why talking? And what if they are right? I can still reinvent myself as a developer, after all.

It was February 2025 when Emanuele, a post doc in our group pointed me at Claude Code, and showed me a demo. Immediately hooked, I fed it with the repo of the simplified KD-SLAM revision. It found four bugs in a teaching prototype: one misplaced parenthesis causing a serious logic error, two latent issues that never fired on the test data, and one cosmetic. Claude unpacked the architecture flawlessly. I felt truly disoriented. Advice: never implement functions you don’t need just because you are at it. A bug will be waiting for you.

Then I wanted to challenge Claude, and wanted to do a “revision” of the good old KD-SLAM-1, by adding the continuous time aspect and polishing rev-1. And I wanted to restart coding. It was brutal, after more than one year out of active development I was looking at the stuff written by old me and was feeling the dumbest creature on earth. It took me 2 good weeks to gear up again. I used Claude in specific sections, and zero in others. The architecture, the plumbing the core algorithms are all organic. The serialization bit and the ROS2-Build-system nightmare are mostly generated.

Yet it happened I wanted to touch the code that was generated, and I was unable to read it. Lambdas called once popping out at every corner, extra long functions. It did the job, and it worked but well, was I able to touch that stuff? Meanwhile I was diving into Transformers, Embeddings, and all the things that the AI repulsion prevented me from looking at, trying to catch up. By now I believe I have a reasonable understanding of the process, even if as a roboticist I’d like to correlate those weights to “something” that I can see and touch. Seeing a solver converge makes me satisfied. When I wrote the solver.

So I changed approach with Claude. Tore down all the serialization it had written, and approached this problem from scratch by directing the generator to develop tiny bits, inspecting them and testing. The most popular prompt was “show

me the call sites of this”. That worked quite well for me. A key rule that emerged from the process was: never ask an LLM to do something you are not able at, and even if you ask, check the results and revise the architecture.

So KD-SLAM was a result of me “restarting”. All this was done on commodity hardware, on a 7-year-old laptop with 32GB of RAM that was cheap back then. Should I sell the RAM now that there is a shortage due to the proliferation of AI data centers – I might become rich. In the future, will we commoners be able to *own* compute that is minimally useful? Even if we can, will we be able to use it? The comfort of delegating reasoning is there. Humans like comfort. Who will own the compute, and hence who will decide who does not pose a threat, and to whom?

As expected KD-SLAM2 worked, since its predecessor did, but compared to it was much more a software engineering work. The core ideas were all in KD-SLAM1. It was FAST. Then I happened to test it on an Apollo lake and with my surprise the CPU version was running faster than real time. The cheap computation could be a selling point that contrasts most of the modern methods. But then I looked at the entire development and thinking process and I realized that the efficiency is a byproduct of the minimalist drive and deterministic thinking that being aware of the power budget imposes. While designing the system I did not have to think about efficiency because inefficient paths were not considered by my thinking process. Instead each time something “symmetric” or “invariant” was arising I thought how to generalize that and what could be built with that property only. This reflects in the brutal satisfaction I feel each time I commit a negative line count. As a final remark I have to thank all the team I belong to for putting up with my complaints and pushing me to come out from the bureaucracy mud.

- Luca Di Giammarino — who proposed the title trailer —
- Barbara Bazzana
- Emanuele Giacomini
- Leonardo Brizi
- Omar Salem
- Lorenzo De Rebotti
- Simone Ferrari
- Davide Ceriola

I am still PhD Director, but I compress these activities on one day of the week, and I managed to install processes that appear to reduce the pain by cutting on context switches and by making my position clear to whoever concerned.

This is the journey of KD-SLAM. Do what you please with it. Even nothing.

References

- [1] P.J. Besl and N.D. McKay. Method for registration of 3-d shapes. In *Sensor fusion IV: control paradigms and data structures*, volume 1611. Spie, 1992.
- [2] Peter Biber and Wolfgang Straßer. The normal distributions transform: A new approach to laser scan matching. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2003.
- [3] Leonardo Brizi, Emanuele Giacomini, Luca Di Giammarino, Simone Ferrari, Omar Salem, Lorenzo De Rebotti, and Giorgio Grisetti. VBR: A Vision Benchmark in Rome. <https://rvp-group.net/>.
- [4] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J.J. Leonard. Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age. *IEEE Transactions on Robotics*, 32(6), 2016.
- [5] Ziyu Cao, William Talbot, and Kailai Li. RESPLE: Recursive spline estimation for LiDAR-based odometry. *IEEE Robotics and Automation Letters*, 10(10), 2025.
- [6] Davide Ceriola, Simone Ferrari, Luca Di Giammarino, Leonardo Brizi, and Giorgio Grisetti. Breaking time: A fully Gaussian framework for distributed and continuous-time SLAM. *IEEE Robotics and Automation Letters*, 2026. to appear.
- [7] B. Della Corte, I. Bogoslavskyi, C. Stachniss, and G. Grisetti. A general framework for flexible multi-cue photometric point cloud registration. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2018.
- [8] Krzysztof Cwian, Luca Di Giammarino, Simone Ferrari, Giorgio Grisetti, Piotr Skrzypczyński, and Thomas Ciarfuglia. MAD-BA: 3D LiDAR bundle adjustment – from uncertainty modelling to structure optimization. arXiv:2501.03972, 2025.

-
- [9] Lorenzo De Rebotto, Emanuele Giacomini, Giorgio Grisetti, and Luca Di Giammarino. Resolution where it counts: Hash-based GPU-accelerated 3D reconstruction via variance-adaptive voxel grids. *ACM Transactions on Graphics*, 45(2), 2025.
 - [10] Pierre Dellenbach, Jean-Emmanuel Deschaud, Bastien Jacquet, and François Goulette. CT-ICP: Real-time elastic LiDAR odometry with loop closure. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2022.
 - [11] J. Deschaud. Imls-slam: scan-to-model matching based on 3d data. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2018.
 - [12] Luca Di Giammarino, Leonardo Brizi, Tiziano Guadagnino, Cyrill Stachniss, and Giorgio Grisetti. MD-SLAM: Multi-cue direct SLAM. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
 - [13] D. Droschel and S. Behnke. Efficient continuous-time slam for 3d lidar-based online mapping. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018.
 - [14] R. Dubé, D. Dugas, E. Stumm, J. Nieto, R. Siegwart, and C. Cadena. Segmatch: Segment based place recognition in 3d point clouds. In *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, 2017.
 - [15] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun. Vision meets robotics: The kitti dataset. *Intl. Journal of Robotics Research (IJRR)*, 32(11), 2013.
 - [16] Emanuele Giacomini, Leonardo Brizi, Tiziano Guadagnino, and Giorgio Grisetti. MAD-ICP: It is all about matching data – robust and informed LiDAR odometry. *IEEE Robotics and Automation Letters (RA-L)*, 9(8), 2024.
 - [17] Emanuele Giacomini, Luca Di Giammarino, Lorenzo De Rebotto, Giorgio Grisetti, and Martin R. Oswald. Splat-LOAM: Gaussian splatting LiDAR odometry and mapping. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
 - [18] L. Di Giammarino, I. Aloise, C. Stachniss, and G. Grisetti. Visual place recognition using lidar intensity information. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2021.
 - [19] G. Grisetti, T. Guadagnino, I. Aloise, M. Colosi, B. Della Corte, and D. Schlegel. Least squares optimization: From theory to practice. *Robotics*, 9(3), 2020.
 - [20] Tiziano Guadagnino, Benedikt Mersch, Nikolas Gupta, Ignacio Vizzo, Giorgio Grisetti, and Cyrill Stachniss. KISS-SLAM: A simple, accurate, and fast LiDAR-based SLAM system with enhanced generalization capabilities. arXiv:2503.12660, 2025.
 - [21] J.-S. Gutmann and K. Konolige. Incremental mapping of large cyclic environments. In *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 1999.
 - [22] Giseop Kim and Ayoung Kim. Scan context: Egocentric spatial descriptor for place recognition within 3D point cloud map. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2018.
 - [23] Cedric Le Gentil, Leonardo Brizi, Daniil Lisus, Xinyuan Qiao, Giorgio Grisetti, and Timothy Barfoot. DRO: Doppler-aware direct radar odometry with gyroscope. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
 - [24] F. Lu and E. Milios. Globally consistent range scan alignment for environment mapping. *Autonomous robots*, 4(4), 1997.
 - [25] A. Nüchter, K. Lingemann, J. Hertzberg, and H. Surmann. Heuristic-based laser scan matching for outdoor 6d slam. In *Annual Conf. on Artificial Intelligence*, 2005.
 - [26] Yue Pan, Xingguang Zhong, Ignacio Vizzo, Louis Liao, Jens Behley, and Cyrill Stachniss. PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency. *IEEE Trans. on Robotics (TRO)*, 40, 2024.
 - [27] Francois Pomerleau, Francis Colas, Roland Siegwart, and Stephane Magnenat. Comparing ICP variants on real-world data sets. *Autonomous Robots*, 34, 2013.
 - [28] A. Segal, D. Haehnel, and S. Thrun. Generalized-icp. In *Proc of Robotics: Science and Systems*, 2009.
 - [29] Jacopo Serafin and Giorgio Grisetti. NICP: Dense normal based point cloud registration. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2015.
 - [30] T. Shan and B. Englot. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, 2018.
 - [31] R. Smith, M. Self, and P. Cheeseman. Estimating uncertain spatial relationships in robotics. In *Uncertainty in Artificial Intelligence*. Elsevier, 1988.

- [32] T. Stoyanov, M. Magnusson, H. Andreasson, and A. J. Lilienthal. Fast and accurate scan registration through minimization of the distance between compact 3d ndt representations. *Intl. Journal of Robotics Research (IJRR)*, 31(12), 2012.
- [33] M. Angelina Uy and G. Hee Lee. Pointnetvlad: Deep point cloud based retrieval for large-scale place recognition. In *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [34] Ignacio Vizzo, Tiziano Guadagnino, Benedikt Mersch, Martin Velas, Cyrill Stachniss, and Jens Behley. KISS-ICP: In defense of point-to-point ICP – simple, accurate, and robust registration if done the right way. *IEEE Robotics and Automation Letters (RA-L)*, 8(2), 2023.
- [35] J. Zhang and S. Singh. Loam: Lidar odometry and mapping in real-time. In *Proc. of Robotics: Science and Systems (RSS)*, 2014.